

AGENTTX: Causal Effect Transactions for Long-Running Coding Agents

Anonymous submission

Abstract

Coding agents execute long trajectories in which later tool calls consume filesystem state produced by earlier exploration, editing, compilation, and testing. Existing command, session, and checkpoint mechanisms either expose unvalidated effects or discard later work that is independent of a failed step. We introduce Agent Effect Transactions (AET), which use causal dependencies among versioned filesystem effects as the recovery unit within one opaque trajectory. AGENTTX realizes AET by capturing typed effects, building a causal ledger, reconstructing affected object versions, and durably publishing an approved frontier. Across 144 controlled runs, AGENTTX retained 100% of independent work and removed 100% of invalid descendants; temporal recovery retained 41% at 64 calls. At 48 document entries, retained artifacts avoided 1,425 replay tokens versus temporal recovery and 3,340 versus whole-session abort. In 50 fresh x86 repeats of a ten-write comparison, the full path measured 58.757 ms/step mean (68.169 ms/step p99) and was the only policy with 50/50 causal-correct recoveries.

1 Introduction

Coding agents increasingly execute long, stateful software-engineering tasks rather than produce a patch in one turn. Their control loops interleave model reasoning with tool actions and observations, allowing a plan to change after a command fails or reveals new repository evidence [15]. SWE-agent and OpenHands expose repository navigation, file editing, shell commands, and tests to the model [11, 14]. SWE-bench likewise evaluates issue resolution as an interactive process that can span several files and test runs [5].

We call such an execution a *multi-step agent task* when each tool call can consume observations or workspace state produced by earlier calls. The agent sees one adaptive trajectory, but the kernel sees process trees whose filesystem effects become visible as they execute. A late test may consume an artifact from an earlier failed attempt, while an independent

edit made between them remains valid. Recovery must keep effects provisional without forcing the agent to replay the entire trajectory.

Existing systems preserve intermediate state at other boundaries. Branch and checkpoint runtimes retain a branch, session, or temporal prefix [3, 10, 13]. `try` stages one opaque command in an unprivileged semisolate and applies selected effects after approval [6]. Agent-facing filesystems, process checkpointing, and transactional filesystems target workspace control, full-session restart, or ACID publication [1, 2, 4, 7, 16]. These recovery units do not select a failed causal subgraph inside one opaque trajectory.

Consider four interleaved steps. Step s_0 writes a faulty formatter, s_1 reads it and produces a report, s_2 edits an independent document, and s_3 tests the faulty output. Recovery should remove $\{s_0, s_1, s_3\}$ while retaining s_2 . No temporal checkpoint necessarily represents that state: a suffix rollback also loses s_2 , while a ledger without read evidence can keep the invalid report.

This recovery-unit mismatch creates three systems challenges. *Dependency discovery* must recover producer and consumer relations hidden by shells, compilers, tests, and third-party programs. *Object identity* must distinguish shared objects from names split by copy-up or replacement across symlinks, hard links, renames, and bind mounts. *Selective reconstruction* must rebuild a non-contiguous state despite repeated writes, deletes, retained suffixes, and interrupted publication.

Existing substrates index provisional state by command, namespace, time, or pathname, while failures propagate through dependencies among versioned effects. We therefore introduce Agent Effect Transactions (AET), which use the trajectory as the speculation unit and a causal effect subgraph as the recovery unit. AGENTTX realizes AET on Linux by capturing READ, NEGATIVE, WRITE, and DELETE effects, maintaining a causal ledger, and reconstructing affected object versions. An approval frontier and write-ahead log protect publication, while persistent workers and incremental snapshots reduce repeated execution cost.

Across 144 controlled runs, causal rollback retained 100% of independent work and removed 100% of invalid descendants. At 64 calls, temporal recovery retained 41% of independent work. At 48 document entries, causal retention avoided 1,425 replay tokens versus temporal recovery and 3,340 versus whole-session abort. These token results measure recovery-induced replay, not total session-token savings.

This paper makes four contributions:

- We formulate causal-subgraph recovery for opaque coding-agent filesystems, preserving independent work that follows a failed step.
- We introduce AET, which represents filesystem effects and their causal dependencies as the basis for selective recovery.
- We build AGENTTX to realize AET with object-aware reconstruction, fail-closed alias handling, durable publication, and efficient persistent execution.
- We evaluate AGENTTX through controlled recovery, latency, replay-token, failure-recovery, identity, and concurrency experiments.

2 Background and Motivation

2.1 Problem Setting and Recovery Units

We model a coding-agent trajectory as an ordered sequence of tool calls $T = \langle s_0, s_1, \dots, s_n \rangle$. A step may invoke a shell, compiler, test runner, package manager, or arbitrary third-party program. Descendant processes can expose effects immediately, and later calls consume the accumulated state. A failed step can leave files, caches, deletions, or diagnostics that affect later execution.

We classify effects as successful reads (R), negative lookups (N), writes (W), and deletes (D). A negative lookup matters when a later write creates the missing path, while directory ancestry affects how that path resolves. Recording only writes misses derived artifacts; treating every touched path as dependent discards independent work. Exit status and temporal order therefore do not identify the correct recovery set.

Paths are not stable object identifiers. Symlinks, hard links, renames, and bind mounts create aliases, while OverlayFS copy-up can split an inode during a write [8]. Recovery must track namespace, object identity, and the version at which each effect was observed.

Isolation and recovery are different boundaries. A per-call sandbox keeps the host clean but repeatedly reconstructs state; a session sandbox amortizes setup but normally aborts the whole trajectory; and a temporal checkpoint discards a suffix. None selects a failed producer and its non-contiguous descendants while retaining independent later work. Table 1 makes these native units explicit; checkpoint and whole-session rows describe semantics rather than external-system executions.

Semisolates stage commands [6]; agent-oriented filesystems and branch runtimes manage workspaces or

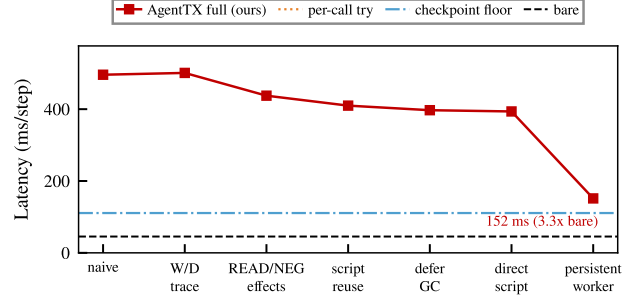


Figure 1: **Amortizing isolation dominates the optimization history.** A *persistent try worker* removes repeated namespace setup and provides the largest endpoint improvement, although the final full path remains slower than unsafe bare execution.

branches [10, 16]; checkpoint systems preserve complete process or session state [1–3, 13]; and transactional filesystems protect ACID publication [4, 7]. Their native units remain commands, sessions, branches, checkpoints, or explicit transactions.

Our target is a continuous speculative view that leaves the host unchanged until approval, removes a failed effect and its descendants selectively, and publishes the approved frontier durably. We consider workspace-local effects from trusted but fallible coding agents. Network and cloud effects, process memory, terminal state, and adversarial confinement remain outside scope.

2.2 Analyzing the Execution Path

Methodology. We run a deterministic 64-call coding workload under six modes. The workload contains repository exploration, a multi-file refactor, an injected failing CI path, a report derived from the faulty file, independent documentation and configuration edits, and a repair suffix. We measure end-to-end wall time per tool call, host cleanliness before approval, and the p50/p95 tails. The full comparison appears later in Table 2; the motivation matrix is deliberately not an unsafe-versus-safe comparison alone, because bare execution is not correctness-equivalent.

Observation. Bare execution is the latency lower bound at 49.7 ms/step but immediately pollutes the host. Per-call try costs 260.7 ms/step, shared try costs 254.0 ms/step, and the shared-checkpoint component costs 63.5 ms/step. Full AGENTTX costs 148.5 ms/step while keeping the host clean; disabling read tracing lowers it to 62.6 ms/step but loses read-derived dependencies. Thus the full traced path is 2.99× bare and per-call isolation is 5.24× bare, even before model/network latency is included. Longer runs amortize fixed setup (156.4, 139.5, and 107.0 ms/step at 54, 64, and 96 calls), but the tail remains large: step p50 ranges from 19.6 to 24.4 ms, versus p95 from 684.8 to 724.0 ms.

Mode	Speculative unit	Native recovery unit	Later independent work	Host before approval
Bare	none	none	survives by immediate mutation	polluted
Per-call semisolate	one call	one call	no continuous shared state	clean
Session semisolate	trajectory	whole session	discarded on abort	clean
Temporal checkpoint	trajectory prefix	suffix after checkpoint	discarded with suffix	clean

Table 1: **Isolation does not determine recovery granularity.** Existing command, session, and temporal units cannot select a failed causal subgraph while retaining non-overlapping later work.

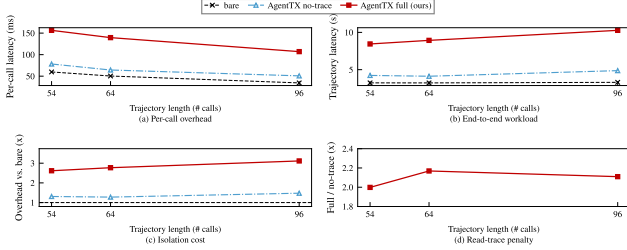


Figure 2: **Longer trajectories amortize setup but do not remove isolation cost.** Across 54, 64, and 96 calls, fixed setup is amortized, yet full AgentTX remains above bare and read tracing adds a measurable penalty. The end-to-end panel shows why per-step latency alone is insufficient for long sessions.

Cost breakdown and attempted fixes. Figure 1 records the hot path before each optimization. Declared effects for trusted tools, reusable command scripts, deferred blob collection, and direct script execution remove smaller repeated costs. A persistent worker reduces 393.6 to 151.5 ms/step by amortizing namespace and overlay setup, while incremental snapshot replay reduces the snapshot stage from 0.384 to 0.158 s. These changes reduce overhead but cannot decide whether a later report depends on a failed producer. The no-op tracing microbenchmark isolates only 23.8 ms/step (8.0%) of additional cost; shell and test workloads exercise more process and path-name events, which explains the much larger tail. The tracer choice is not the semantic fix: read capture adds runtime cost, but the recovery predicate depends on the resulting effect edges rather than on a particular tracer implementation.

2.3 Analyzing the Recovery Path

Methodology. We execute real filesystem effect DAGs rather than simulated graph nodes. The sweep varies trajectory length (16/32/64), shape (chain, fan-out, layered), fault position (10/50/75%), and independent-work ratio (25/50/75%). Causal, temporal, and whole-session policies receive the same trajectory, while a fourth mode removes read dependencies. Three repeats yield 144 real-overlay runs over 48 configurations, and every run checks both the speculative view and the physical host before commit.

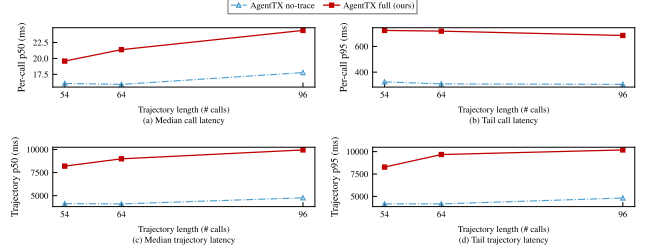


Figure 3: **Tracing widens tail latency in long-agent execution.** The full path has substantially larger call and trajectory p50/p95 than the no-trace ablation at every tested length. This motivates reporting tail distributions and separating tracing optimizations from recovery correctness.

Observation. Full causal rollback retains 100% of independent work and removes 100% of invalid descendants. Temporal rollback removes invalid work but retains only 41% of independent work at 64 calls; whole-session abort retains 0%. The no-dependency ablation retains apparent files but removes only 4% of invalid work. In the long workload, the fault roots are steps [27, 29, 30]; full tracing finds all three and removes the derived report, whereas no-trace finds only step 27 and leaves that report behind.

Implication. The temporal policy loses 59% of independent work before any model is asked to repair it. In the token experiment, this corresponds to 1,424.7 avoided replay tokens against an optimistic temporal checkpoint and 3,340.3 against whole-session abort at 48 document entries. These are recovery-induced replay tokens, not total session-token savings. The result is a semantic gap, not a constant-factor implementation cost: faster isolation cannot compensate for discarding work that was already correct.

2.4 Design Requirements

The measurements rule out four shortcuts. Rebuilding isolation per call costs $5.24\times$ bare execution and fragments trajectory state. Shared temporal state reduces setup cost but still discards independent edits, while disabling read tracing misses compiler and test dependencies. Path-only tracking also remains insufficient: the tested hard-link cases pass, but bind mounts, external aliases, and arbitrary generation

changes fail closed. The WAL recovers interrupted publication but does not provide external all-path atomic visibility.

Existing substrates index provisional state by command, namespace, time, or pathname, while failures propagate through causal dependencies among versioned effects. The trajectory must therefore be the speculation unit, and the causal subgraph must be the recovery unit. Dependency capture, versioned object identity, and selective reconstruction must work together. A precise rollback policy over an incomplete graph or unsupported identity relation is unsafe.

3 Agent Effect Transactions

3.1 Model and Goals

An Agent Effect Transaction is a tuple $\langle V, L, H, F \rangle$. V is the current speculative filesystem view. L records tool steps and their effects, while H provides historical states needed for reconstruction. F separates finalized steps from the speculative suffix. A tool call reads and mutates V , appends one node to L , and leaves the host unchanged.

The four components map directly to the goals in Section 2. V preserves continuous cross-call state while isolating the host. L identifies a causal recovery set, and H reconstructs it without discarding an independent suffix. F publishes only approved effects. AET defines these transitions without requiring a particular tracing, snapshot, or commit implementation.

An AET exposes three transitions. *Append* executes one tool call in V and adds its effects to L . *Recover* selects a failed node, computes its descendant closure, and reconstructs the affected portion of V from H . *Finalize* advances F and publishes only the approved versions. These transitions obey three invariants: the host is unchanged before finalization; successful recovery removes every selected effect while preserving every non-overlapping retained effect; and finalized steps never become speculative again. The invariants clarify the contribution boundary: AET specifies what must hold, while AgentTX supplies one Linux implementation.

3.2 Effect DAG

Each ledger node $s_i = (i, \text{tool}_i, E_i, P_i, \text{status}_i)$ contains effects and parent step identifiers. Let $\text{overlap}(p, q)$ hold when two effect names identify overlapping objects. For an active earlier step $j < i$, the transaction adds $j \in P_i$ when:

- s_j writes or deletes p , and s_i reads, negatively looks up, writes, or deletes an overlapping q .
- s_j negatively looks up p , and s_i later writes an overlapping q .

The first rule captures read-after-write and write serialization. The second captures assumptions based on absence. The identity relation is part of the AET contract; Section 4 states how AgentTX approximates it.

We make identity explicit with an abstract relation $\equiv_I$ over effect names. Path equality implies overlap, but aliases may also overlap when the relation can justify it. A read through a file descriptor is attached to the object opened earlier when the observation layer supplies that association; otherwise the effect is unknown rather than guessed. This separation prevents the ledger from silently turning incomplete observation into a false dependency.

For a failed active step f , let $C_0 = \{f\}$. We compute

$$C_{k+1} = C_k \cup \{s_i \mid P_i \cap C_k \neq \emptyset\}, \quad C(f) = \bigcup_{k \geq 0} C_k. \quad (1)$$

$C(f)$ is the least transitive descendant closure among active, uncommitted steps. Temporal rollback instead selects the suffix $\{s_i \mid i \geq f\}$.

3.3 Selective Reconstruction

Let $W(C)$ contain paths written or deleted by steps in $C(f)$. For each target, the runtime obtains a state that predates its earliest selected effect. It then checks every retained active effect against $W(C)$. If a retained effect overlaps a target, the runtime rejects recovery rather than guessing a merge. This fail-closed rule trades availability for a safety invariant:

After a successful causal rollback, no effect written by $C(f)$ is visible, and every non-overlapping retained effect remains visible in the speculative view.

The transaction marks selected nodes rolled back only after reconstruction succeeds. Historical states for invalidated steps can then be reclaimed.

3.4 Commit Frontier

The frontier F is the largest finalized step identifier. Every step at or below F is approved for publication or durably marked rolled back. Advancing to $F' > F$ publishes active write/delete effects in $(F, F']$ and leaves later steps speculative. Rolled-back holes therefore do not block monotonic progress.

Publication and speculative recovery are separate transitions. An implementation must publish the frontier version even if a later speculative step rewrites the same object. AET requires durable recovery of that transition, but not atomic multi-path visibility to external observers.

4 AgentTX Design and Implementation

4.1 Tool-Boundary Effect Capture

Writes and deletes are obtained by fingerprinting the unmounted upper layer before and after a command. Fingerprints include content and metadata, thereby capturing repeated `chmod`, `chown`, `touch`, `empty-directory`, `symlink`, and

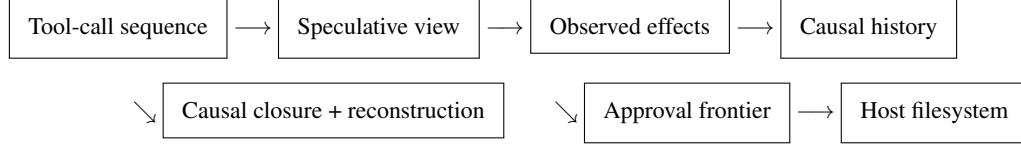


Figure 4: **Agent Effect Transaction lifecycle.** A trajectory shares one speculative view, causal history selects recovery, and an approval frontier controls publication.

whiteout effects. Linux OverlayFS may encode deletions as character-device or `.wh.*` whiteouts; both forms are normalized to D effects.

AgentTX captures successful reads and ENOENT/ENOTDIR lookups with `strace` [9]. The parser follows process creation, tracks working directories, and resolves modeled path operations relative to file descriptors. It records requested and resolved paths when a symlink changes the name. A failed trace invocation or parse aborts the step unless tracing is explicitly disabled. Coverage remains limited to modeled Linux pathname syscalls; it does not prove that every filesystem access is observed. Trusted tools may declare effects directly, making their declarations part of the trusted computing base.

An optional persistent eBPF backend emits the same effect contract from syscall tracepoints and uses the same parser semantics. The main evaluation uses `strace`, whose coverage and latency were more stable on our host; the backend choice does not change the AET contract. For a relative path through a non-`AT_FDCWD` descriptor, either backend requires a resolved descriptor origin; if the tracer cannot provide one, the tool call fails closed instead of treating the caller’s working directory as that origin.

The capture contract distinguishes three cases. A path-based open or negative lookup contributes a directly named R or N effect. A descriptor-relative operation contributes an effect only when the trace contains a resolvable descriptor origin; inherited or duplicated descriptors without such an origin remain outside the supported identity contract. A trusted declaration can fill this gap for a structured tool, but the declaration is an assumption, not an independently verified observation. This fail-closed boundary keeps the causal ledger conservative instead of fabricating edges from command text.

4.2 Shared Semisolate and Persistent Worker

All calls in one session reuse a `try -N` upper layer. This shared layer provides the continuous speculative view required by AET. Later commands observe earlier uncommitted effects while the host remains unchanged. The launcher synchronizes `$PWD` with the protected workspace because a changed subprocess `cwd` alone can leave the environment inconsistent.

The persistent Python worker is a performance optimization over that design. It exchanges length-prefixed requests inside one semisolate to amortize namespace and overlay

setup. If the worker dies, the current request uses the one-shot path; the following request starts a new worker. This fallback preserves availability without making worker reuse part of AET correctness.

4.3 Snapshots and Aliases

Before step i , the layer store records `before_i`. Regular files are stored as immutable content-addressed blobs and hard-linked into snapshots. After the first step, an incremental snapshot clones the prior tree and replays only paths changed by the previous call. Native whiteouts and mode-000 trees are preserved. Causal rollback restores only selected logical paths; temporal rollback swaps the full upper tree to one pre-step image.

AgentTX treats ancestor-related paths as overlapping and records lexical and resolved paths for supported symlink accesses. Renames appear as path-level delete/create effects; the session catalog supplies stable object identifiers for known hard-link groups, but not for all namespace changes. Bind-mount equivalence is outside the identity model.

The supported identity levels are therefore deliberately ordered. Directory ancestors are conservatively overlapping, symlink accesses retain both names, and rename is represented as a delete/create pair. Hard-link relations are promoted only when the substrate or the upperdir materializer verifies the object identity; bind-mount relations remain fail-closed. This policy may reject a recovery that could be safe, but it avoids claiming precision that path-level evidence cannot provide.

The active OverlayFS path enables `index=on`, preserving hard-link visibility during live copy-up. After one name is written, an in-transaction read through its sibling observes the new bytes. Before publication, AgentTX discovers complete host-visible alias groups and expands both frontier and retained-suffix checks. It updates the linked host inode in place instead of renaming one upper file over one alias. A persisted object catalog connects a write through one known name to a read through its sibling. The publication path also handles tested upper-created hard-link groups, distinguishes copy-up from intentional rename-over replacement, and rebases a retained suffix onto a fresh upper generation. Bind mounts, external aliases, arbitrary generation changes, and general multi-object publication remain outside this contract and fail closed.

4.4 Durable Commit and Policy

AgentTX uses anchored include filters to materialize selected frontier paths. If a later speculative step rewrites one path, AgentTX reconstructs the frontier version, publishes it, and restores the later upper-layer state. The runtime evaluates the persisted policy before WAL preparation or host mutation.

Before publication, the WAL saves affected host paths, the upper layer, the target frontier, and prior ledger state. Reload either restores the pre-commit state or finalizes a durable frontier. Session metadata uses same-directory atomic replace followed by file and directory fsync.

The publication path has three observable phases. *Prepare* records the frontier and the host values needed for recovery. *Install* materializes the selected historical versions under the policy filter. *Finalize* records the new frontier and releases obsolete snapshots. A crash before finalize leaves the old frontier recoverable; a crash after finalize reloads the new frontier. This state machine explains what the WAL contributes without turning it into a claim of external multi-path atomicity.

A shared allow/deny policy is enforced by the runtime, CLI, coding harness, and LLM agent. Policy rules persist across reload. The WAL provides crash recoverability, not external all-path atomic visibility: a concurrent observer can see a prefix while materialization is in progress.

5 Evaluation

Our evaluation answers five questions:

- RQ1** Does causal recovery remove invalid descendants while retaining independent work (Section 5.2)?
- RQ2** What runtime cost do trajectory isolation and dependency capture add (Section 5.3)?
- RQ3** Which capture, identity, reconstruction, and publication mechanisms are necessary (Section 5.4)?
- RQ4** Can a live agent use causal recovery, and how many replay tokens does retained work avoid (Section 5.5)?
- RQ5** Do worker failure, reload, long sessions, and concurrency preserve the transaction invariants (Section 5.6)?

5.1 Setup and Methodology

Experiments run on an x86₆₄ server with an AMD EPYC 7713, 32 online CPUs, 62 GiB of memory, and Linux 5.4.0-216-generic. The prototype uses upstream `try`, OverlayFS, `strace`, and Python 3.8.10. Live-agent experiments use `deepseek-v4-flash`.

Every workload starts from a fresh workspace. Correctness validators inspect the speculative view and host state before recovery, after recovery, and after controlled commit. The fixed comparison uses 50 independent samples per mode; the causal and token sweeps use three repeats per aggregated configuration. The live recovery experiment uses three fresh

sessions, while the GitHub-context suite uses one run per task and policy.

We call a run *causal-correct* when the host is clean before publication, recovery removes the invalid producer and every derived artifact, and the independent artifact remains visible. Modes without causal recovery can still measure execution cost, but they do not satisfy this joint predicate.

We separate baselines by the property they measure. Bare execution is an unsafe latency lower bound because it writes to the host immediately. Per-call `try`, session `try`, shared `try`, shared checkpoints, and Bubblewrap isolate different execution costs but do not implement causal recovery. The no-trace configuration removes read dependencies and serves as a correctness ablation. Temporal checkpoint and whole-session abort emulate coarser recovery over the same trajectory; they are not external-system runs.

5.2 RQ1: Causal Recovery Semantics

We hypothesize that a causal recovery unit can retain later independent work while removing a failed producer and its descendants. A temporal recovery unit should remove the corruption but discard unrelated suffix work. Removing dependency capture should instead preserve derived corruption that appears independent to the ledger.

The sweep executes filesystem effects through the real overlay rather than simulating graph nodes. Its 48 aggregated configurations span lengths of 16, 32, and 64; chain, fan-out, and layered DAGs; three fault positions; and three independent-work ratios. Causal, temporal, and whole-session recovery receive identical declared reads, while the fourth mode removes read dependencies. Three repeats produce 144 runs, each validated for host cleanliness, independent retention, and invalid-descendant removal.

Figure 5 reports both retention and invalid removal. Full causal recovery reaches 100% on both metrics in every tested configuration. At 64 calls, temporal recovery removes the invalid work but retains only 41% of independent work; whole-session abort retains none. The no-dependency ablation retains independent files but removes only 4% of invalid descendants.

Causal rollback has a p95 latency of 272.7 ms at 64 calls. The separate ten-write predicate also succeeds in 50 of 50 full-system runs and zero of 50 runs for every other measured mode. These experiments validate the recovery-unit claim on controlled effect DAGs, not arbitrary repository failures or non-filesystem effects.

5.3 RQ2: Runtime Cost and Scaling

We hypothesize that persistent shared execution amortizes the setup paid by per-call isolation. Full dependency capture should cost more than the no-trace ablation because it observes reads and retains a larger effect graph. Unsafe bare

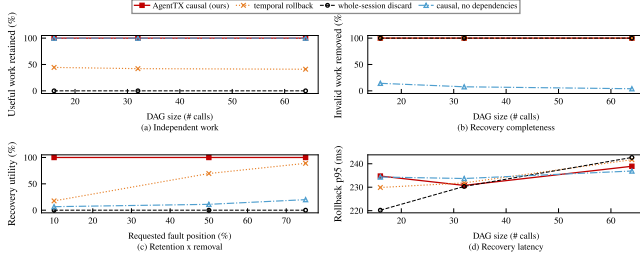


Figure 5: Causal recovery needs dependency capture. Across 144 controlled runs, full recovery retains and removes 100%; at 64 calls, temporal recovery retains 41%, and the ablation removes only 4% of invalid work.

Table 2: Runtime on the 64-call coding workload. Full AgentTX is 43% faster than per-call `try`, but remains $2.99\times$ slower than unsafe bare execution.

Mode	ms/step	Host clean
Bare	49.7	no
Per-call <code>try</code>	260.7	yes
Shared <code>try</code>	254.0	yes
Shared snapshot	63.5	yes
AGENTTX no-trace	62.6	yes
AGENTTX full	148.5	yes

execution remains a latency lower bound, not a correctness-equivalent baseline.

Table 2 reports the 64-call coding workload. Full AGENTTX costs 148.5 ms/step, 43% less than per-call `try` at 260.7 ms/step. It remains $2.99\times$ slower than bare execution at 49.7 ms/step, which pollutes the host before commit. The 85.9 ms/step difference from no-trace includes both system-call tracing and the additional dependency state.

We separately measure runtime distributions on a fixed ten-write trajectory. Each mode runs 50 times on fresh workspaces with `strace` selected for the full configuration. Table 3 reports the distribution, and Figure 6 pairs p99 latency with the causal predicate. These values are not compared numerically with the 64-call workload.

Longer trajectories show the effect of fixed setup costs. At 54, 64, and 96 calls, full AGENTTX costs 156.4, 139.5, and 107.0 ms/step. Total wall time still rises from 8.45 to 10.27 s, so lower per-step cost does not mean a faster longer run. Step p50 ranges from 19.6 to 24.4 ms, while p95 ranges from 684.8 to 724.0 ms.

The repeated ten-write experiment establishes a stable distribution only for that fixed trajectory. The 64-call and scaling results use different command mixes, so their absolute values are not interchangeable. Across both studies, persistent execution lowers repeated isolation cost, but tracing and heavy tool boundaries still dominate the tail.

5.4 RQ3: Mechanism and Publication Evidence

We hypothesize that causal recovery needs all three core mechanisms: typed effect capture, object-aware reconstruction, and durable publication. The mechanism suite combines a 64-call dependency validator, a three-repeat 20-step trace microbenchmark, three policy entry points, and commit failures on both sides of host installation. Each test checks the corresponding ledger, reconstructed view, frontier, and host state.

The 64-call dependency validator injects fault roots at steps [27, 29, 30]. Full AGENTTX identifies all three, removes the faulty formatter and derived report, and retains the independent documentation, configuration, and repair. Without read tracing, the ledger identifies only step 27 and leaves the report. This result connects the 4% ablation result in RQ1 to a concrete missed edge.

The trace microbenchmark isolates capture cost from tool behavior. Enabling tracing raises mean latency from 295.63 to 319.43 ms/step across three repeats, an increase of 23.80 ms or 8.0%. Shell and test commands access more paths, so this no-op result is not a workload-wide tracing estimate.

Identity tests cover parent-child paths, negative lookups, symlink aliases, and supported hard-link groups. With `OverlayFS index=on`, a lower hard-link write is visible through its sibling inside the speculative view. Selective commit expands a complete pre-existing alias group and publishes both names through one inode with link count two. Tested upper-created groups and intentional rename-over replacement also pass. Bind mounts, external aliases, and arbitrary generation changes remain outside the publication contract.

Publication tests apply one deny rule through the direct API, CLI, and a reloaded session. All three reject a denied `*.pem` effect before WAL preparation, leaving both the frontier and host unchanged. Failure injection before and after host installation either restores the prior frontier or resumes the recorded finalized frontier. This WAL provides deterministic recovery, not atomic visibility to an external observer during materialization.

Content-addressed snapshots store 9.1 MB of unique blobs for 100.7 MB of logical snapshot payload, a 9.0% ratio. Regression tests also cover native whiteouts, mode-000 trees, metadata restoration, and interrupted materialization. The storage result does not remove directory traversal, historical snapshot, or WAL-copy costs.

5.5 RQ4: Live-Agent Recovery and Avoided Replay

We hypothesize that a tool-selecting model can use the causal ledger without a scripted rollback target. We also expect retained artifacts to avoid the model calls needed to regenerate work lost by coarse recovery. The first experiment tests

Table 3: **Runtime distribution on the fixed ten-write trajectory.** Full AgentTX reaches 68.169 ms/step at p99, 66.4% below per-call try at 202.746 ms/step, and satisfies the causal predicate in all 50 runs.

Mode	Mean	p50 ms/step	p95	p99	Causal-correct runs/50
Bare	2.164	2.095	2.323	4.976	0/50
Per-call try	193.726	193.826	201.427	202.746	0/50
Session try	18.621	18.232	21.988	22.390	0/50
Shared try	184.230	184.204	190.712	196.776	0/50
Shared checkpoint	47.186	46.599	51.206	52.147	0/50
Bubblewrap	1.145	1.162	1.274	1.301	0/50
AGENTTX no-trace	50.395	49.818	54.100	55.508	0/50
AGENTTX full	58.757	57.738	63.511	68.169	50/50

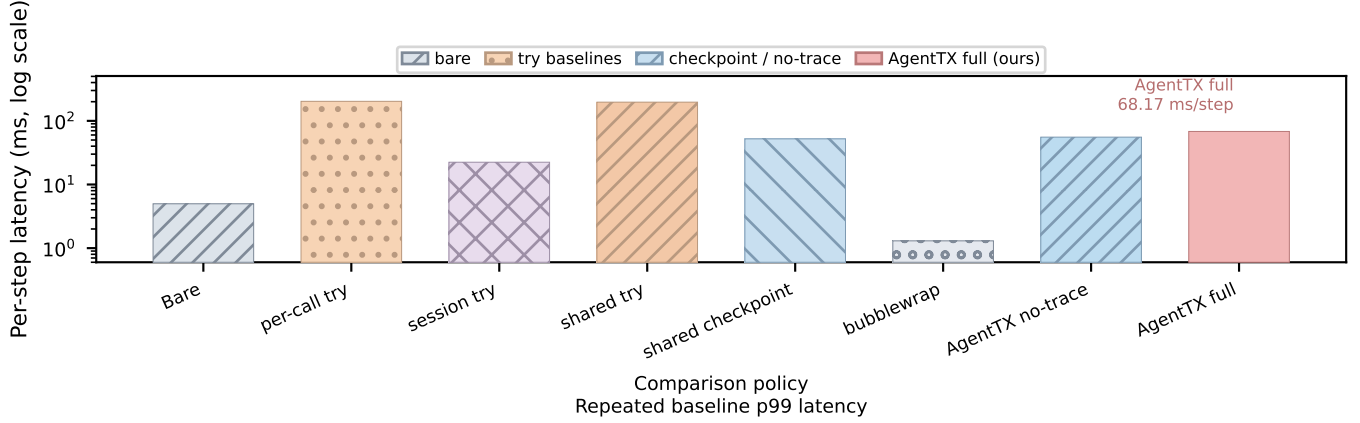


Figure 6: **Persistent execution reduces the p99 isolation cost.** Across 50 ten-write runs, full AgentTX reaches 68.169 ms/step, 66.4% below per-call try, and is causal-correct in every run.

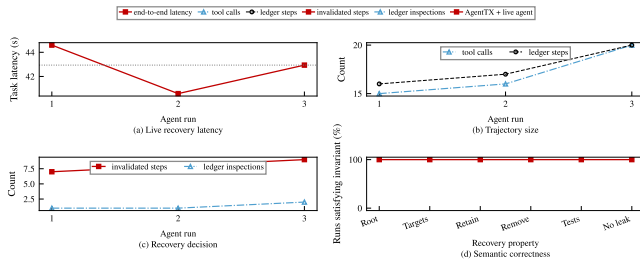


Figure 7: **A live agent uses causal recovery.** In all three sessions, the agent selects the faulty root, retains the independent artifact, removes invalid descendants, passes tests, and avoids a pre-commit host leak.

control-plane use; the second isolates replay caused by recovery granularity.

The live benchmark seeds one repository with a faulty pipeline, an independent release note, a derived invalid artifact, and failing tests. The model must inspect the ledger, choose the earliest faulty producer, invoke causal rollback, retain the note, remove the artifact, repair the task, and pass tests. The validator then checks host cleanliness and performs the controlled commit. Figure 7 summarizes three fresh sessions.

Root selection, rollback target, independent retention, invalid removal, and tests succeed in all three sessions. Host

leakage remains zero, and wall-time p50/p95 is 42.9/44.4 s with deepseek-v4-flash. This experiment covers one seeded task and one model, not general success on large repositories.

Avoided replay tokens. The controlled token study uses a five-step trajectory with two valid documents interleaved with one faulty producer and its descendants. Causal recovery retains both documents, an optimistic temporal checkpoint loses one, and whole-session abort loses both. A real model regenerates missing documents at 12, 24, and 48 entries, with three repeats per policy and size. The validator checks document structure, tests, rollback target, and pre-commit host state.

Figure 8 reports all 27 samples. Every sample succeeds without a host leak. Causal recovery needs no regeneration call and avoids between 864 and 1,425 tokens relative to temporal recovery. It avoids between 1,797 and 3,340 tokens relative to whole-session abort.

These values are avoided replay tokens, not total session-token savings. Temporal and whole-session policies are recovery-granularity emulations, not external artifacts. The experiment isolates the model work caused by losing valid files and excludes shared diagnosis, validation, and pre-failure tokens.

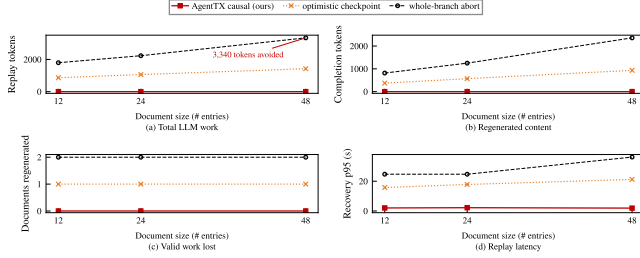


Figure 8: **Retained artifacts avoid controlled LLM replay.** At 48 entries, temporal recovery replays 1,424.7 tokens and whole abort replays 3,340.3; zero excludes diagnosis, tests, and pre-failure work.

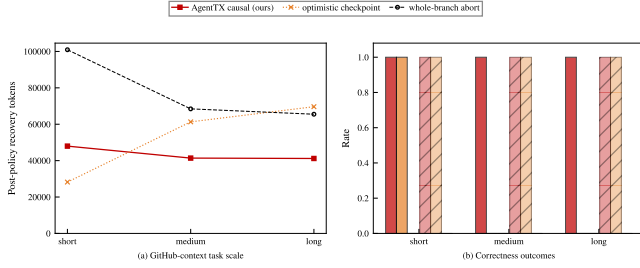


Figure 9: **Exploratory recovery cost in repository context.** Each point is one run; causal recovery succeeds on all three tasks, while token counts from failed coarse-policy runs are not savings.

We also run nine exploratory policy-task pairs on pinned Requests, Flask, and pytest snapshots. Each task exposes repository context but separates its contract, solution, tests, independent documents, and derived artifact into recovery directories. Causal recovery satisfies tests and retention in all three tasks; temporal recovery succeeds only on the short task; whole-session abort succeeds on none. Figure 9 pairs tokens with these success outcomes.

On the short task, temporal recovery succeeds with 28,209 tokens, compared with 47,977 for causal recovery. On the medium and long tasks, only causal recovery satisfies the complete success predicate. Because each cell has one run and success differs across policies, this suite provides exploratory context evidence rather than a statistical token comparison.

5.6 RQ5: Failure Recovery and Session Scale

We hypothesize that the performance optimizations preserve the transaction invariants after worker loss, reload, and concurrent execution. The robustness bundle injects one persistent-worker failure, reloads one 256-step session at step 128, and runs four agents with 16 writes each. Validators check completed files, frontier state, host leakage, and cross-workspace contamination.

Figure 10 summarizes the three tests. The killed worker falls back to one-shot `try` for the current request and restarts before the next. The reloaded session commits all 256 files

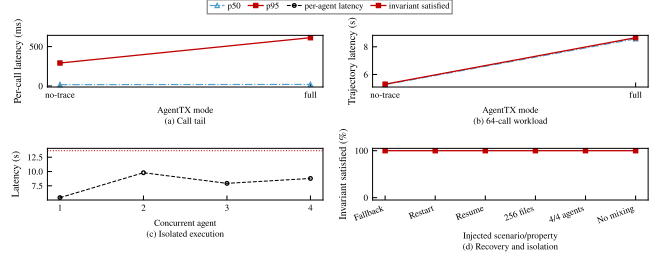


Figure 10: **The optimized runtime preserves tested invariants.** Worker fallback, a 256-step reload, and four disjoint agents all complete without losing committed files or contaminating another workspace.

with step p50/p95 of 36.3/72.3 ms. All four agents commit their disjoint subtrees without cross-contamination in 3.105 s.

The worker and reload tests exercise recovery within one session, while the concurrency test uses separate workspace subtrees. The latter does not test fencing for concurrent writes to the same path. RQ5 therefore validates the tested failure and isolation cases, not arbitrary overlapping multi-agent publication.

6 Discussion and Limitations

Why not make causal rollback the default? The active view uses OverlayFS `index=on`, which preserves hard-link visibility within the tested session [8]. Alias-complete publication covers complete host-visible groups, tested upper-created groups, and intentional rename-over replacement. Causal recovery still fails closed for arbitrary multi-object topology, bind mounts, external aliases, and unsupported generation changes.

Why trace instead of infer commands? Command strings are not a reliable specification of transitive compiler, test, or script reads. System-call tracing observes actual accesses but adds workload-dependent cost and remains Linux-specific. We therefore use the trace path for the main correctness and performance claims: in the measured 12-step comparison it costs 25.60 ms/step, versus 61.39 ms/step for the optional persistent eBPF backend, while both capture the same read and negative lookup cases in this probe. The eBPF path remains a replaceable implementation, not a claim of lower overhead. Neither tracer removes the need to define alias and unsupported-syscall semantics or repairs an inode relation already split by OverlayFS copy-up.

What state is outside the transaction? The current AET covers workspace-local filesystem effects. Network requests, emails, package registries, cloud control planes, process memory, and terminal state may be irreversible or require subsystem-specific compensation. Systems that capture full

process/session state are complementary, not dominated by AGENTTX.

Is commit atomic? The WAL guarantees deterministic recovery of the session and host after an interrupted commit. It does not provide kernel-level all-path atomic visibility to external observers during materialization. Workloads requiring that property need a rename-based publication protocol or a transactional substrate.

How strong is the current evidence? Deterministic workloads cover up to 256 steps and controlled DAGs up to 64 calls. Real-agent runs are seeded and use one model. Several recent external systems cannot run on the current VM because of FUSE, CRIU, kernel, compiler, or privilege requirements. A final submission needs broader repositories, interleaved statistical runs, and artifact-level comparisons where available.

7 Related Work

AgentTX is closest to effect-capturing semisolates, agent-oriented filesystems, and checkpoint or branch runtimes. Its distinguishing recovery unit is a non-contiguous causal subgraph within one speculative trajectory. We compare that semantic choice below; without artifact-native measurements, we do not claim end-to-end superiority.

Effect capture and semisolates. `try` introduces unprivileged semisolates for opaque commands, including effect introspection, optional application, and stacking [6]. AGENTTX uses these mechanisms as its execution substrate. Its focus is the long-lived trajectory: automatic cross-step dependencies, non-contiguous causal recovery, frontier materialization, policy persistence, and commit WAL.

Agent-native filesystems and branches. YoloFS stages mutations, exposes snapshots, and uses progressive permission to improve agent/user control [16]. Its filesystem mediation and permission interface are broader than AGENTTX’s current substrate. BranchFS provides nested copy-on-write branch contexts with atomic leaf commit/abort and first-commit-wins coordination [10]. These systems select snapshots or branches; AGENTTX selects a causal subgraph inside one evolving trajectory.

Checkpoint and restore. Waypoint combines filesystem deltas with process, memory, shell, and terminal state for branchable sessions [13]. DeltaBox accelerates complete sandbox checkpoint/rollback using incremental filesystem and process state [3]. Crab observes turn-level effects to avoid checkpoints that do not contain recovery-relevant state [12].

Their state coverage or checkpoint efficiency is complementary to AGENTTX. AGENTTX instead answers which non-contiguous filesystem effects to remove after a failure has been identified.

Process checkpointing and file transactions. CRIU and DMTCP preserve complete process/session state for restart, migration, or debugging [1, 2]; their recovery unit is an application image, not a selected filesystem effect subgraph. Valor exports POSIX-like file transactions with kernel logging and locking, while TxFS builds ACID file transactions on filesystem journaling [4, 7]. These systems target atomic publication and crash consistency for explicit transactions. AgentTX instead keeps the existing unmodified application interface, infers effects at opaque tool boundaries, and preserves valid later work after a selective failure.

Filesystem topology and tracing. OverlayFS specifies copy-up, whiteout, and index semantics that determine when two pathnames still denote one object [8]. System-call tracers such as `strace` observe accesses made by arbitrary descendants [9]. The resulting paths still need hierarchy, descriptor, and alias normalization. Our contribution is to connect that observation layer to a causal ledger and a fail-closed reconstruction policy; we do not claim that a different tracer by itself solves object identity or selective publication.

8 Conclusion

Long-running coding agents need a transaction boundary larger than one command and a recovery unit smaller than one session. An Agent Effect Transaction selects a causal subgraph of versioned effects instead of a temporal suffix. AGENTTX realizes this abstraction with shared speculation, an effect ledger, selective reconstruction, and a durable commit frontier. In 144 controlled runs, it retained all independent work and removed all invalid descendants; at 48 document entries, it also avoided measurable LLM replay. The result is limited to supported workspace-local filesystem effects and fail-closed object identity.

References

- [1] Jason Ansel, Kapil Arya, and Gene Cooperman. DMTCP: Transparent checkpointing for cluster computations and the desktop, 2009.
- [2] CRIU developers. Checkpoint/restore in userspace, 2026.
- [3] Yunpeng Dong, Jingkai He, Shiqi Liu, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Baochuan Yang, Yubin Xia, and Haibo Chen. Deltabox: Scaling stateful ai agents with millisecond-level sandbox checkpoint/rollback, 2026.
- [4] Yige Hu, Zhiting Zhu, Ian Neal, Youngjin Kwon, Tianyu Cheng, Vijay Chidambaram, and Emmett Witchel. TxFS:

- Leveraging file-system crash consistency to provide ACID transactions. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 879–892. USENIX Association, 2018.
- [5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
 - [6] Evangelos Lamprou, Tianyu (Ezri) Zhu, Di Jin, Grigoris Ntousakis, Georgios Liargkovas, Calvin Eng, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Controlling Opaque-Component effects with semisolates and try. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 453–471. USENIX Association, 2026.
 - [7] Richard P. Spillane, Sachin Gaikwad, Manjunath Chinni, Erez Zadok, and Charles P. Wright. Enabling transactional file access via lightweight kernel extensions. In *7th USENIX Conference on File and Storage Technologies (FAST 09)*, pages 219–232. USENIX Association, 2009.
 - [8] The Linux Kernel Developers. Overlay filesystem. Linux kernel documentation, 2026.
 - [9] The strace developers. strace: Linux system call tracer, 2026.
 - [10] Cong Wang and Yusheng Zheng. Fork, explore, commit: Os primitives for agentic exploration, 2026.
 - [11] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [12] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes, 2026.
 - [13] Jiakai Xu, Tianle Zhou, Danielle Gillai, Ruizhe Fu, Georgios Liargkovas, Patrick Shen, Kostis Kaffes, and Eugene Wu. Toward systems foundations for agentic exploration. In *Systems for Agentic AI Workshop at SOSP*, 2025.
 - [14] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
 - [15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [16] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don’t let ai agents yolo your files: Shifting information and control to filesystems for agent safety and autonomy, 2026.